AIT
Asian Institute of Technology

IoT Data Platform Relational Database Design



PRESENTER: KHOA PHAM

Agenda

- Ideation
- Business Model
- Objective
- Conceptual ERD
- Entity Schema
- Data Dictionary
- Business rule and constraint
- Insert/Update operation
- Sample Queries for answer business questions



AIT
Asian Institute of Technology

IDEATION

Providing the Iot Data platform services to track and trace farm’s information, operation and performance. Moreover, the readiness of data quality for advanced data analytics use cases in the future.

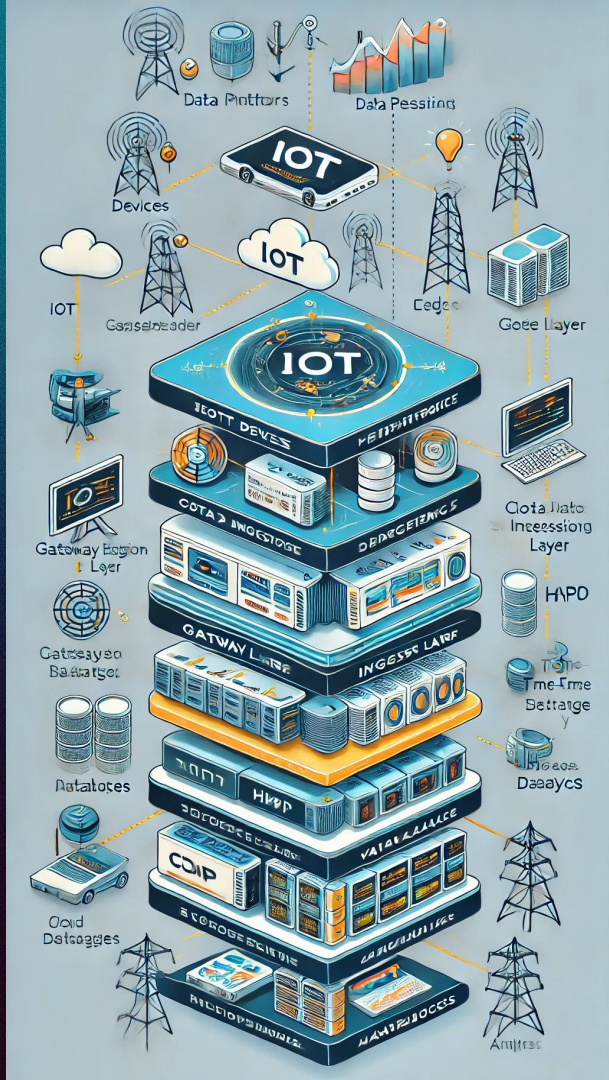


Business Model

- We offer an end-to-end IoT platform with real-time/historical data insights, device management, and seamless integrations.

- We monetize through a subscription-based model by a customers who are joining our IoT platform, charging for the number of devices, and support, while also providing insightful and meaningful from entry to advanced data use cases consulting and custom integration services.

- Customers lower operating costs, boost efficiency, and make faster, data-driven decisions. By centralizing device management and providing real-time analytics, businesses can quickly respond to issues, streamline operations, and unlock new valuable opportunities.





OBJECTIVE

Real-time visibility

Collects up-to-the-minute sensor data (e.g., soil moisture, temperature) for immediate insight into farm conditions.

Informed Decision-Making

Aggregated and historical sensor data supports smarter planning—such as optimal irrigation schedules or feed adjustments.

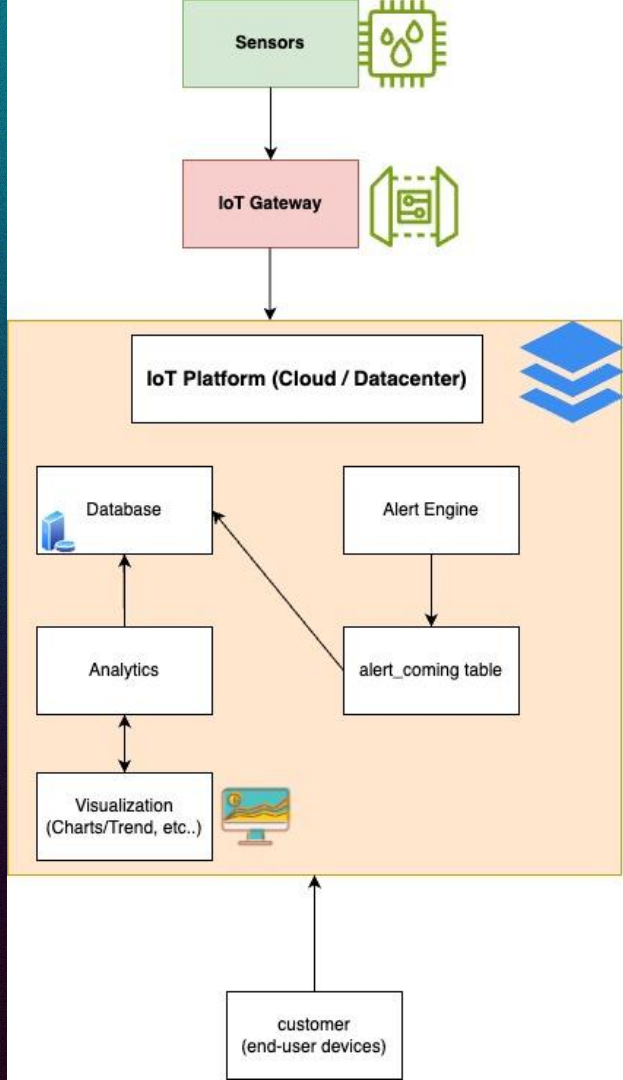
Scalability

Easily add new sensors, farms, data streams as the operation grows

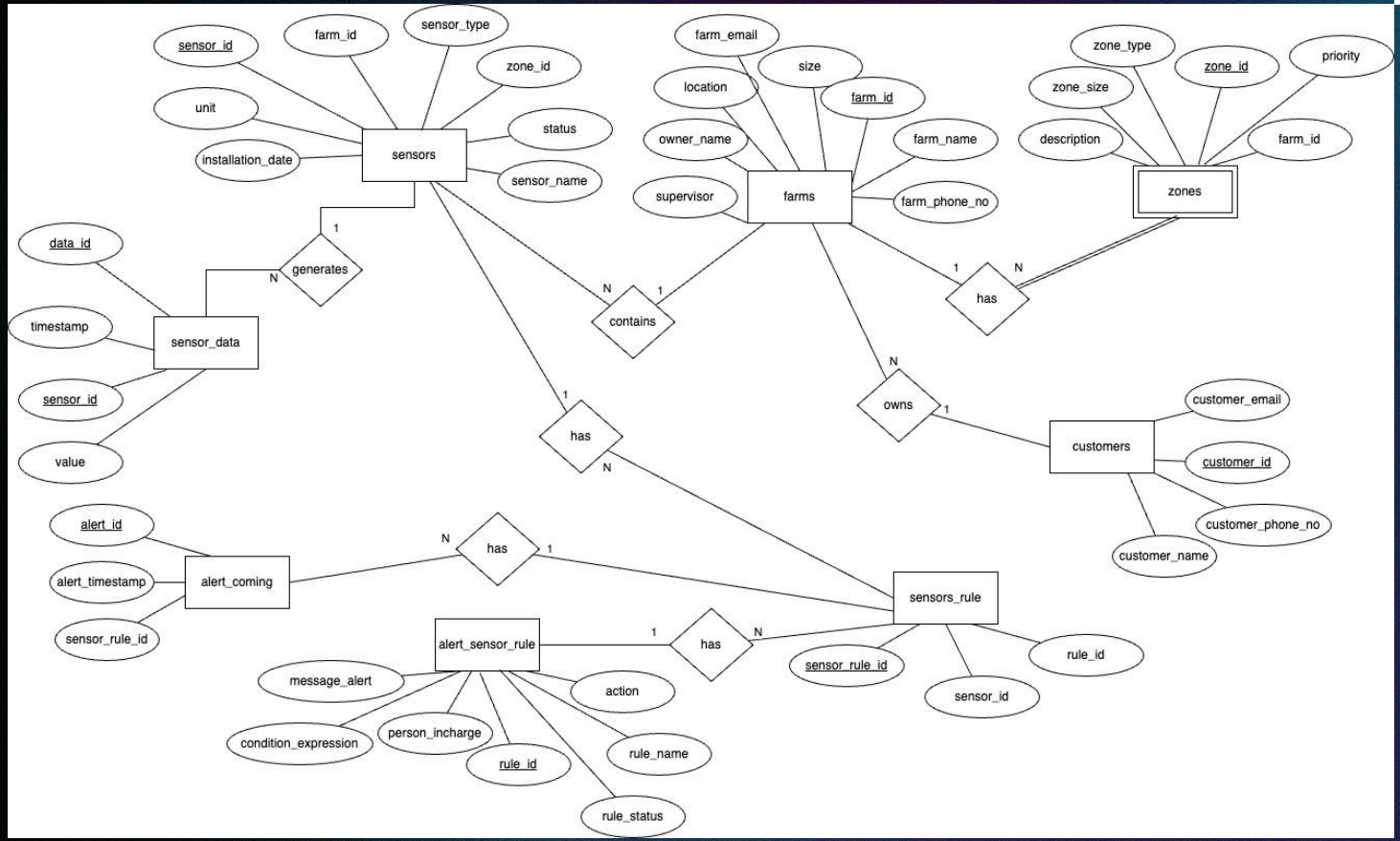
Alert & Rule-based Action

Customize rules - trigger alarms or automated actions when thresholds are met or exceeded

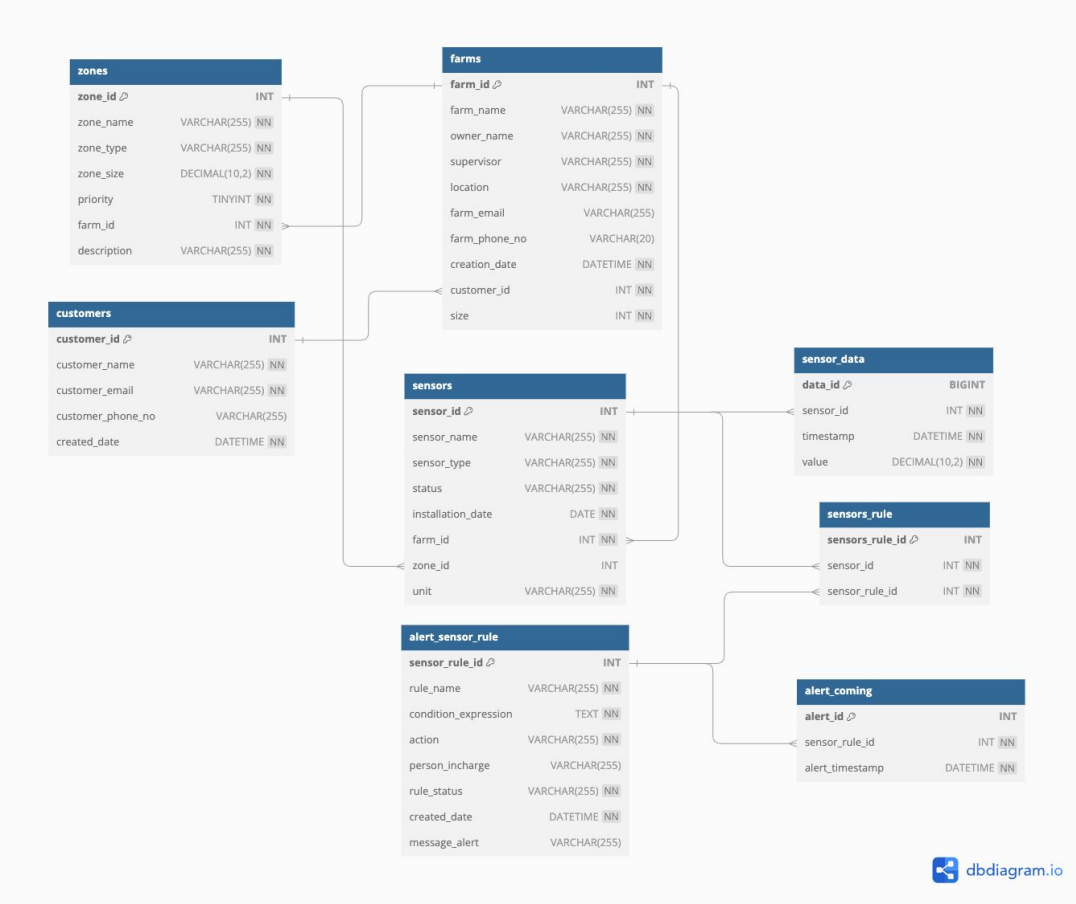
High Level of Data architecture of IoT Platform



CONCEPTUAL ERD



ENTITY SCHEMA



Business Rules

1. Customers ↔ Farms

- One customer can own many farms (1:N)
- Each farm references a valid customer.

2. Farms ↔ Zones:

- One farm can have many zones (1:N).
- Each zone requires a valid farm_id.
- zone_size must be positive

3. Farms ↔ Sensors

- One farm can have many sensors (1:N).
- Each sensor must have a valid farm_id.

Business Rules

4. Sensors ↔ Sensor_Data

- A sensor can generate many data records (1:N).
- timestamp is required, defaults to current time.
- value must match the sensor's unit

5. Sensors_Rule (Linking Sensors to Rules)

- A sensor can have many rules; a rule can apply to many sensors (M:N).
- Each link (sensors_rule) references a valid sensor and a valid rule.
- rule_status (active/inactive) determines if the rule is applied.

6. Alert_Sensor_Rule & Alert_Coming

- One alert_sensor_rule can generate many alerts (1:N).
- Each alert_coming references a valid alert_sensor_rule.
- alert_timestamp defaults to creation time.
- person_incharge shows who handles alerts.
- Inactive rules do not generate new alerts.

Data Dictionary & Schema Creation

Table	Column	Data Type	Default Value	Constraints	Description
customers	customer_id	INT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each customer.
customers	customer_name	VARCHAR(255)		NOT NULL	Full name of the customer (business or individual).
customers	customer_email	VARCHAR(255)		NOT NULL, UNIQUE	Customer's email address (unique per record).
customers	customer_phone_no	VARCHAR(255)	NULL	(none)	Optional phone number for the customer.
customers	created_date	DATETIME	CURRENT_TIMESTAMP	NOT NULL	Date/time record was created.

```sql
-- ========================================
-- 1) CREATE TABLE: customers
-- ========================================
CREATE TABLE `customers` (
  `customer_id` INT AUTO_INCREMENT PRIMARY KEY,
  `customer_name` VARCHAR(255) NOT NULL,
  `customer_email` VARCHAR(255) NOT NULL,
  `customer_phone_no` VARCHAR(255) DEFAULT NULL,
  `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

  UNIQUE KEY (`customer_email`)
);
```

Table	Column	Data Type	Default Value	Constraints	Description
farms	farm_id	INT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each farm.
farms	farm_name	VARCHAR(255)		NOT NULL	Name of the farm (e.g., "Green Valley Farm").
farms	owner_name	VARCHAR(255)		NOT NULL	Name of the farm's owner.
farms	supervisor	VARCHAR(255)		NOT NULL	The person supervising daily operations.
farms	location	VARCHAR(255)		NOT NULL	Geographic location of the farm.
farms	farm_email	VARCHAR(255)	NULL	(none)	Optional farm email address.
farms	farm_phone_no	VARCHAR(20)	NULL	(none)	Optional farm phone number.
farms	creation_date	DATETIME	CURRENT_TIMESTAMP	NOT NULL	Date/time record was created.
farms	customer_id	INT		NOT NULL, FK → customers(customer_id), ON UPDATE CASCADE	Links the farm to its owner in customers.
farms	size	INT		NOT NULL	Size of the farm in chosen units (acres, hectares, etc.).

```sql
-- 2) CREATE TABLE: farms
-- ========================================
CREATE TABLE `farms` (
  `farm_id` INT AUTO_INCREMENT PRIMARY KEY,
  `farm_name` VARCHAR(255) NOT NULL,
  `owner_name` VARCHAR(255) NOT NULL,
  `supervisor` VARCHAR(255) NOT NULL,
  `location` VARCHAR(255) NOT NULL,
  `farm_email` VARCHAR(255) DEFAULT NULL,
  `farm_phone_no` VARCHAR(20) DEFAULT NULL,
  `creation_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `customer_id` INT NOT NULL,
  `size` INT NOT NULL,

  CONSTRAINT `fk_farm_customer`
    FOREIGN KEY (`customer_id`)
    REFERENCES `customers`(`customer_id`)
    ON UPDATE CASCADE
);
```

Table	Column	Data Type	Default Value	Constraints	Description
zones	zone_id	INT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each zone.
zones	zone_name	VARCHAR(255)		NOT NULL	Name of the zone.
zones	zone_type	VARCHAR(255)		NOT NULL	Category/type of zone (e.g., “Field,” “Greenhouse”).
zones	zone_size	DECIMAL(255,2)		NOT NULL	Size of the zone.
zones	priority	TINYINT	1	NOT NULL	Priority level (1 = default).
zones	farm_id	INT		NOT NULL, FK → farms(farm_id), ON UPDATE CASCADE	Indicates which farm this zone belongs to.
zones	description	VARCHAR(255)		NOT NULL	Additional zone details.

```
-- ========================================
-- 3) CREATE TABLE: zones
-- ========================================
CREATE TABLE `zones` (
  `zone_id` INT AUTO_INCREMENT PRIMARY KEY,
  `zone_name` VARCHAR(255) NOT NULL,
  `zone_type` VARCHAR(255) NOT NULL,
  `zone_size` DECIMAL(10,2) NOT NULL,
  `priority` TINYINT NOT NULL DEFAULT 1,
  `farm_id` INT NOT NULL,
  `description` VARCHAR(255) NOT NULL,

  CONSTRAINT `fk_zone_farm`
    FOREIGN KEY (`farm_id`)
    REFERENCES `farms`(`farm_id`)
    ON UPDATE CASCADE
);
```

Table	Column	Data Type	Default Value	Constraints	Description
sensors	sensor_id	INT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each sensor.
sensors	sensor_name	VARCHAR(255)		NOT NULL	Descriptive name (e.g., “Moisture Sensor #1”).
sensors	sensor_type	VARCHAR(255)		NOT NULL	Type of sensor (e.g., “Temperature,” “SoilMoisture”).
sensors	status	VARCHAR(255)		NOT NULL	Operational status (e.g., “Active,” “Inactive,” “Maintenance”).
sensors	installation_date	DATE		NOT NULL	Date the sensor was installed.
sensors	farm_id	INT		NOT NULL, FK → farms(farm_id), ON UPDATE CASCADE	Links sensor to a specific farm.
sensors	zone_id	INT	NULL	FK → zones(zone_id), ON UPDATE CASCADE	If assigned, sensor is placed in a specific zone.
sensors	unit	VARCHAR(255)		NOT NULL	Measurement unit for the sensor data (e.g., °C, %).

```sql
-- ========================================
-- 4) CREATE TABLE: sensors
-- ========================================
CREATE TABLE `sensors` (
  `sensor_id` INT AUTO_INCREMENT PRIMARY KEY,
  `sensor_name` VARCHAR(255) NOT NULL,
  `sensor_type` VARCHAR(255) NOT NULL,
  `status` VARCHAR(255) NOT NULL,
  `installation_date` DATE NOT NULL,
  `farm_id` INT NOT NULL,
  `zone_id` INT DEFAULT NULL,
  `unit` VARCHAR(255) NOT NULL,

  CONSTRAINT `fk_sensor_farm`
    FOREIGN KEY (`farm_id`)
    REFERENCES `farms`(`farm_id`)
    ON UPDATE CASCADE,

  CONSTRAINT `fk_sensor_zone`
    FOREIGN KEY (`zone_id`)
    REFERENCES `zones`(`zone_id`)
    ON UPDATE CASCADE
);
```

Table	Column	Data Type	Default Value	Constraints	Description
sensor_data	data_id	BIGINT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each sensor reading.
sensor_data	sensor_id	INT		NOT NULL, FK → sensors(sensor_id), ON UPDATE CASCADE	Identifies which sensor generated the data.
sensor_data	timestamp	DATETIME	CURRENT_TIMESTAMP	NOT NULL	When the data was recorded.
sensor_data	value	DECIMAL(10,2)		NOT NULL	Recorded reading value.

```sql
-- ========================================
-- 5) CREATE TABLE: sensor_data
-- ========================================
CREATE TABLE `sensor_data` (
  `data_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
  `sensor_id` INT NOT NULL,
  `timestamp` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `value` DECIMAL(10,2) NOT NULL,

  CONSTRAINT `fk_sensordata_sensor`
    FOREIGN KEY (`sensor_id`)
    REFERENCES `sensors`(`sensor_id`)
    ON UPDATE CASCADE
);
```

Table	Column	Data Type	Default Value	Constraints	Description
alert_sensor_rule	sensor_rule_id	INT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each rule.
alert_sensor_rule	rule_name	VARCHAR(255)		NOT NULL	Human-readable rule identifier.
alert_sensor_rule	condition_expression	TEXT		NOT NULL	Logical condition (e.g., "Value < 30 AND Unit='Moisture%'").
alert_sensor_rule	action	VARCHAR(255)		NOT NULL	Action triggered (e.g., "Send Alert," "Open Valve").
alert_sensor_rule	person_incharge	VARCHAR(255)	NULL	(none)	Who is responsible for this rule.
alert_sensor_rule	rule_status	VARCHAR(255)	"active"	NOT NULL	Whether the rule is active or inactive.
alert_sensor_rule	created_date	DATETIME	CURRENT_TIMESTAMP	NOT NULL	Date/time rule was created.
alert_sensor_rule	message_alert	VARCHAR(255)	NULL	(none)	Additional message or note for when the rule triggers.

```sql
-- ========================================
-- 6) CREATE TABLE: alert_sensor_rule
-- ========================================
CREATE TABLE `alert_sensor_rule` (
  `sensor_rule_id` INT AUTO_INCREMENT PRIMARY KEY,
  `rule_name` VARCHAR(255) NOT NULL,
  `condition_expression` TEXT NOT NULL,
  `action` VARCHAR(255) NOT NULL,
  `person_incharge` VARCHAR(255) DEFAULT NULL,
  `rule_status` VARCHAR(255) NOT NULL DEFAULT 'active',
  `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `message_alert` VARCHAR(255) DEFAULT NULL
);
```

Table	Column	Data Type	Default Value	Constraints	Description
sensors_rule	sensors_rule_id	INT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each sensor-rule link.
sensors_rule	sensor_id	INT		NOT NULL, FK → sensors(sensor_id), ON DELETE CASCADE	Identifies which sensor is tied to this rule.
sensors_rule	sensor_rule_id	INT		NOT NULL, FK → alert_sensor_rule(sensor_rule_id), ON DELETE CASCADE	Identifies which rule applies to this sensor.
sensors_rule	link_status	VARCHAR(20)	"active"	NOT NULL	Status of the link (e.g., “active,” “inactive”).

```
-- ========================================
-- 7) CREATE TABLE: sensors_rule (M:N link)
-- ========================================
CREATE TABLE `sensors_rule` (
  `sensors_rule_id` INT AUTO_INCREMENT PRIMARY KEY,
  `sensor_id` INT NOT NULL,
  `sensor_rule_id` INT NOT NULL,

  CONSTRAINT `fk_sensors_rule_sensor`
    FOREIGN KEY (`sensor_id`)
    REFERENCES `sensors`(`sensor_id`)
    ON UPDATE CASCADE,

  CONSTRAINT `fk_sensors_rule_rule`
    FOREIGN KEY (`sensor_rule_id`)
    REFERENCES `alert_sensor_rule`(`sensor_rule_id`)
    ON UPDATE CASCADE
);
```

Table	Column	Data Type	Default Value	Constraints	Description
alert_coming	alert_id	INT	AUTO_INCREMENT	PK, NOT NULL	Primary key; unique ID for each alert instance.
alert_coming	sensor_rule_id	INT		NOT NULL, FK → alert_sensor_rule(sensor_rule_id), ON UPDATE CASCADE	The rule that triggered the alert.
alert_coming	alert_timestamp	DATETIME	CURRENT_TIMESTAMP	NOT NULL	When the alert was created.

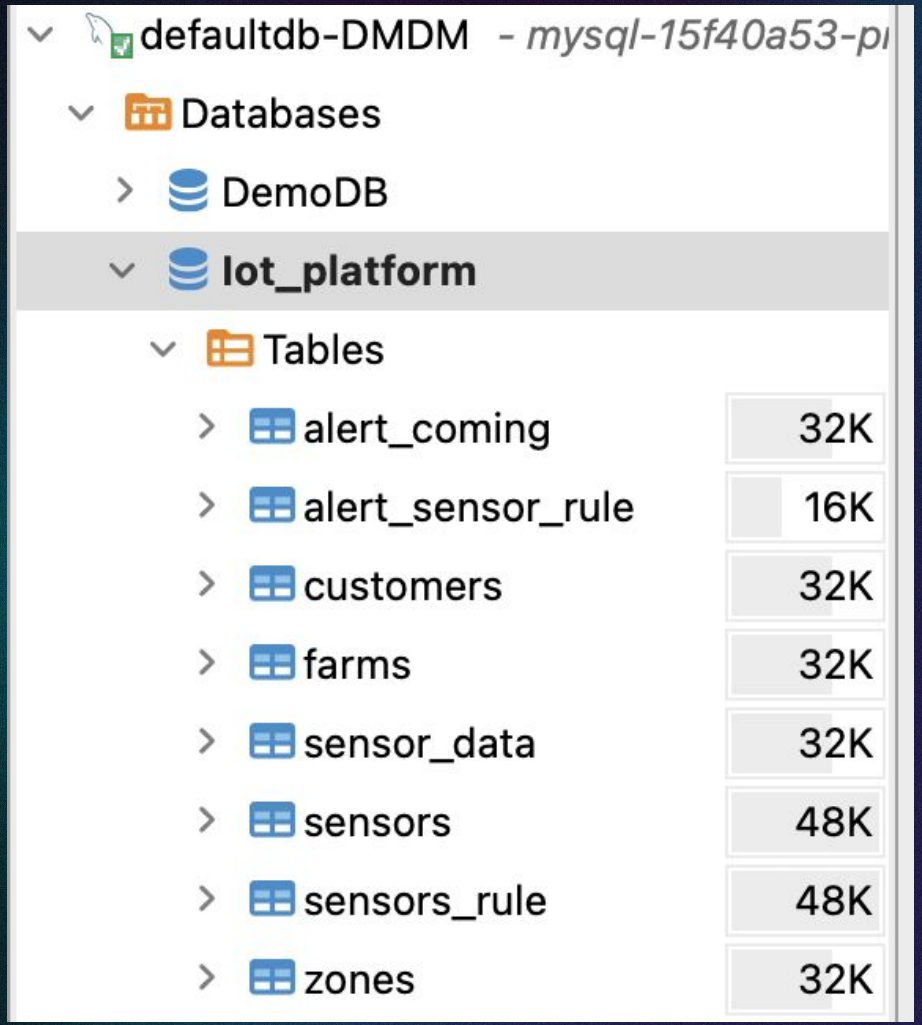
```
-- ========================================
-- 8) CREATE TABLE: alert_coming
-- ========================================
CREATE TABLE `alert_coming` (
  `alert_id` INT AUTO_INCREMENT PRIMARY KEY,
  `sensor_rule_id` INT NOT NULL,
  `alert_timestamp` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT `fk_alert_rule`
    FOREIGN KEY (`sensor_rule_id`)
    REFERENCES `alert_sensor_rule`(`sensor_rule_id`)
    ON UPDATE CASCADE
);
```

Iot_platform

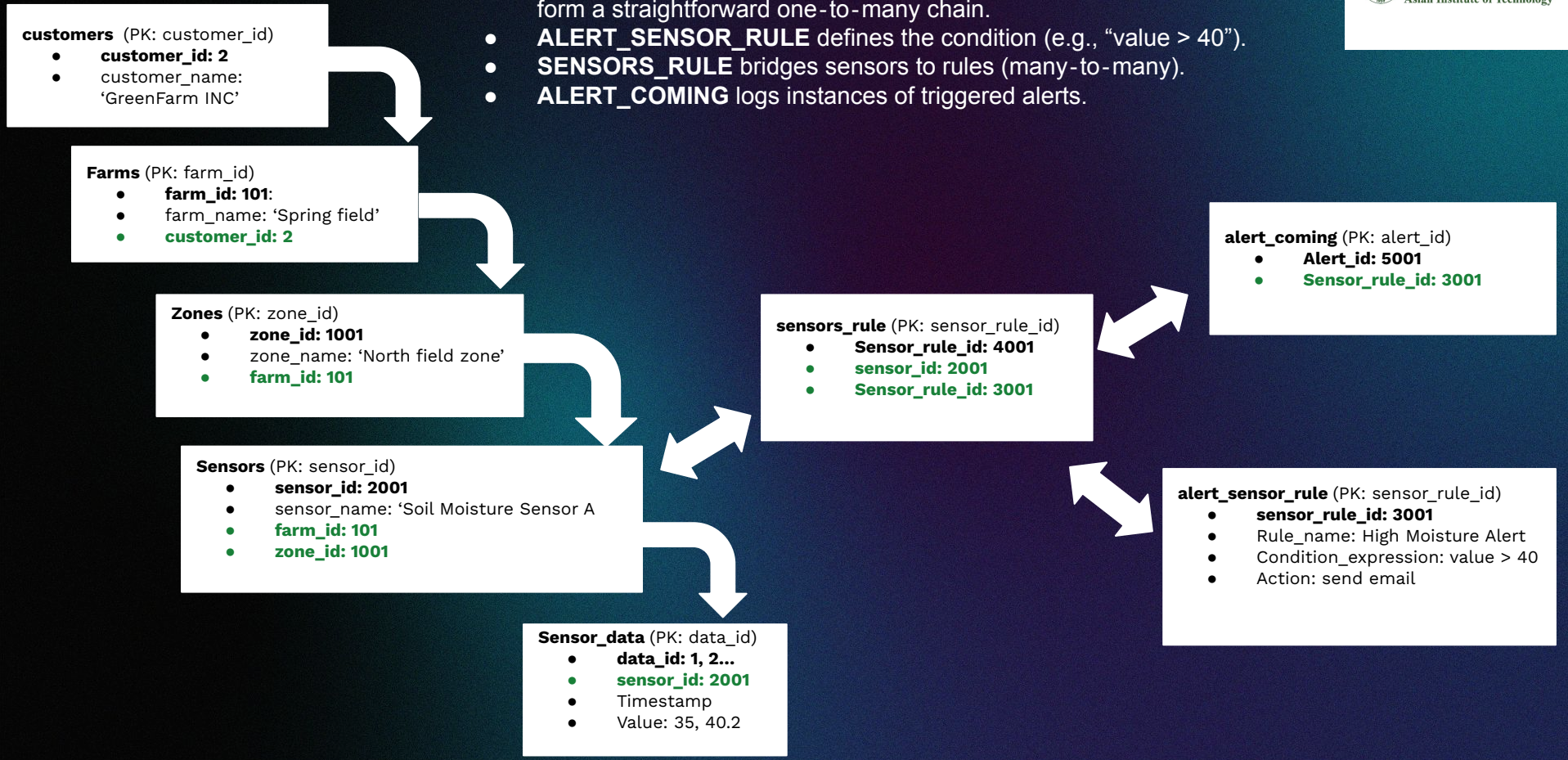
database

schema



Insert New Data

- **CUSTOMERS → FARMS → ZONES → SENSORS → SENSOR_DATA** form a straightforward one-to-many chain.
- **ALERT_SENSOR_RULE** defines the condition (e.g., “value > 40”).
- **SENSORS_RULE** bridges sensors to rules (many-to-many).
- **ALERT_COMING** logs instances of triggered alerts.



Insert New Data

AIT Asian Institute of Technology

```sql
-- 1) Insert a customer
INSERT INTO customers (customer_id, customer_name, customer_email, customer_phone_no, created_date)
VALUES (2, 'GreenFarm Inc', 'info@greenfarm.com', '123-456-7890', NOW());

-- 2) Insert a farm referencing that customer
INSERT INTO farms (farm_id, farm_name, owner_name, supervisor, customer_id, location, creation_date, size,
VALUES (101, 'Spring Field', 'Khoa', 'ABC', 2, 'Riverside Area', NOW(), 120, 'khoa_abc@farm.com', '1234556

-- 3) Insert a zone referencing the farm
INSERT INTO zones (zone_id, zone_name, zone_type, zone_size, priority, farm_id, description)
VALUES (1001, 'North Field Zone', 'Open Field', 50.5, 1, 101, 'Used for main crops');

-- 4) Insert a sensor referencing the farm and zone
INSERT INTO sensors (sensor_id, sensor_name, sensor_type, status, installation_date, farm_id, zone_id, uni
VALUES (2001, 'Soil Moisture Sensor A', 'Moisture', 'Active', '2025-01-01', 101, 1001, 'Percent');

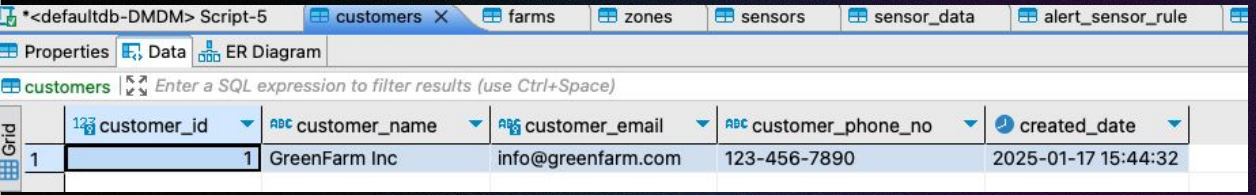
-- 5) Insert some sensor data
INSERT INTO sensor_data (data_id, sensor_id, timestamp, value)
VALUES (1, 2001, NOW(), 35.0),
       (2, 2001, DATE_ADD(NOW(), INTERVAL -1 DAY), 40.2);

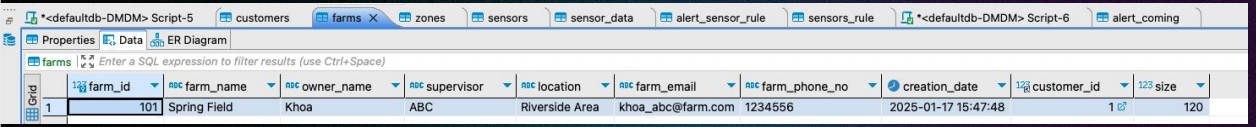
-- 6) Insert a rule in alert_sensor_rule (the definition of a threshold or condition)
INSERT INTO alert_sensor_rule (sensor_rule_id, rule_name, condition_expression, action, person_incharge, r
VALUES (3001, 'High Moisture Alert', 'value > 40', 'Send Email', 'John Doe', 'active', NOW(), 'Moisture le

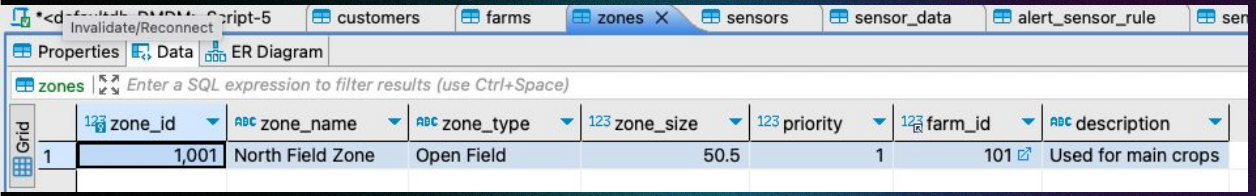
-- 7) Link that rule to the sensor via sensors_rule (M:N relationship)
INSERT INTO sensors_rule (sensors_rule_id, sensor_id, sensor_rule_id)
VALUES (4001, 2001, 3001);

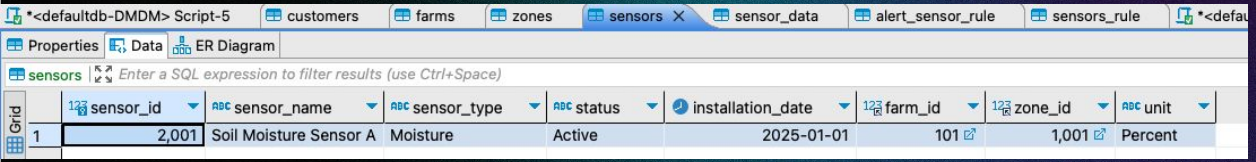
-- Step 8: Insert into alert_coming (triggered alert)
INSERT INTO alert_coming (alert_id, sensor_rule_id, alert_timestamp)
VALUES (5001, 3001, NOW());
```

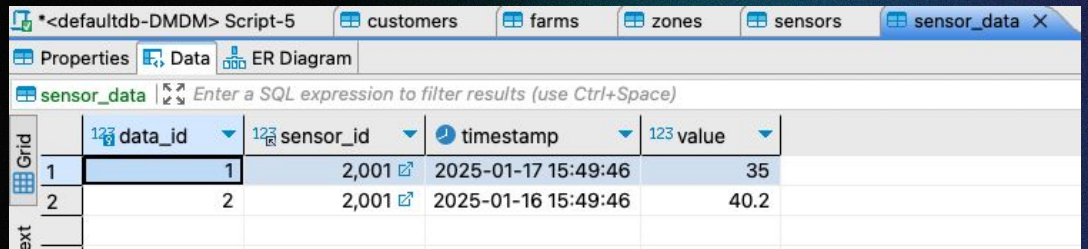
Check inserted data



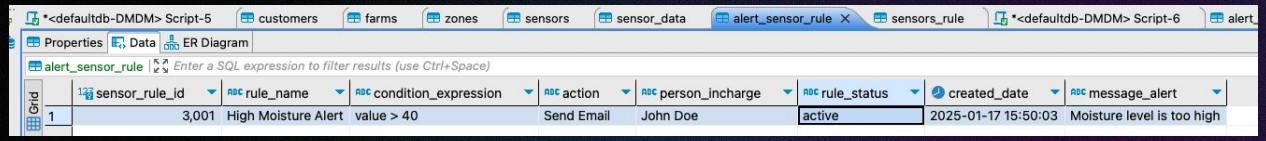




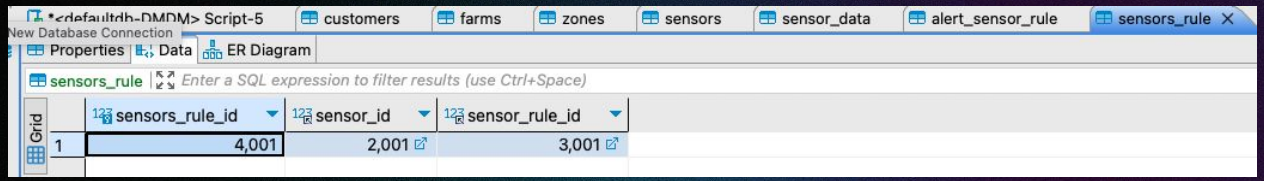




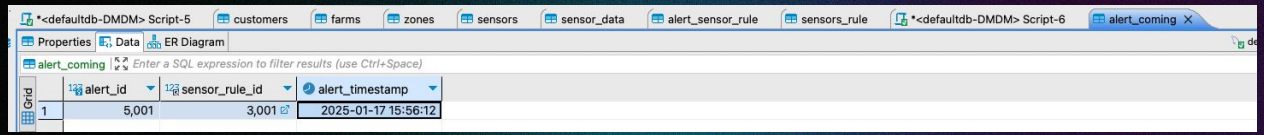
Check inserted data



sensor_rule_id	rule_name	condition_expression	action	person_incharge	rule_status	created_date	message_alert
3,001	High Moisture Alert	value > 40	Send Email	John Doe	active	2025-01-17 15:50:03	Moisture level is too high

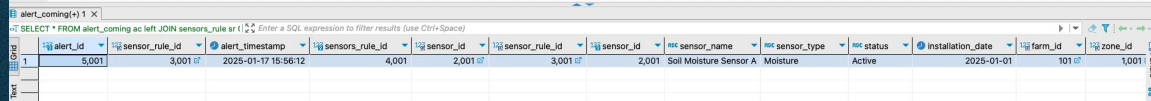


sensors_rule_id	sensor_id	sensor_rule_id
4,001	2,001	3,001



alert_id	sensor_rule_id	alert_timestamp
5,001	3,001	2025-01-17 15:56:12

```
SELECT
*
FROM alert_coming ac
left JOIN sensors_rule sr ON ac.sensor_rule_id = sr.sensor_rule_id
left JOIN sensors s ON sr.sensor_id = s.sensor_id
left JOIN alert_sensor_rule asr ON sr.sensor_rule_id = asr.sensor_rule_id
where rule_status = 'active'
```



alert_id	sensor_rule_id	alert_timestamp	sensors_rule_id	sensor_id	sensor_rule_id	sensor_id	sensor_name	sensor_type	status	installation_date	farm_id	zone_id
5,001	3,001	2025-01-17 15:56:12	4,001	2,001	3,001	2,001	Soil Moisture Sensor A	Moisture	Active	2025-01-01	101	1,001

Check schema integrity after inserting data

Alert coming from which sensor and details of alert

Check inserted data on Visualization tool



AIT Asian Institute of Technology

Green Farm IoT Data Platform

Customer Information

Field	1
customer_id	2
customer_name	GreenFarm Inc
customer_email	info_company@greenfarm
customer_phone_no	123-456-7890
created_date	1737128672000

Farm information

Field	1
farm_id	101
farm_name	Spring Field
owner_name	Khoa
supervisor	ABC
location	Riverside Area
farm_email	khoa_abc@farm.com
farm_phone_no	1234556
creation_date	1737128868000
customer_id	2
size	120

Zones

zone_id	zone_name	zone_type	zone_size	priority	farm_id	description
1001	North Field Zone	Open Field	50.5	1	101	Used for main crops

sensors

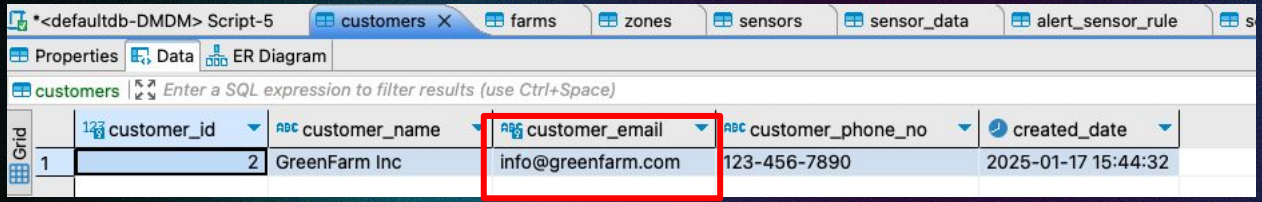
sensor_id	sensor_name	sensor_type	unit	status	installation_date
2001	Soil Moisture Sensor A	Moisture	Percent	Active	2025-01-01 07:00:00
3001	Energy Consumption Counter	Energy	kWh	Active	2025-01-01 07:00:00
4001	Wind Speed Sensor	Wind Speed	km/h	Active	2025-02-01 07:00:00
5001	Greenhouse CO2 Sensor	CO2	ppm	Active	2025-03-01 07:00:00

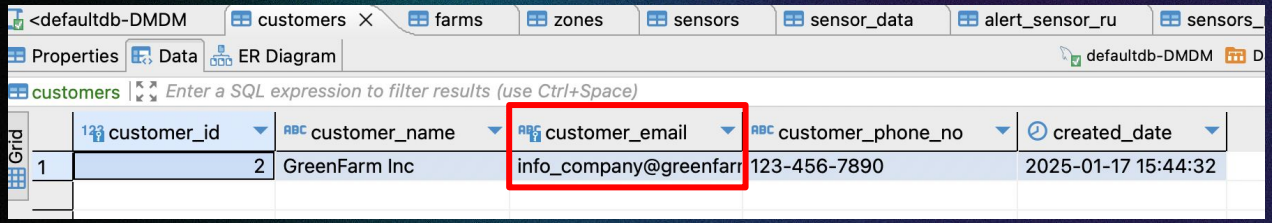
Hight sensors installation location

farm_id	farm_name	zone_id	zone_name	total_sensors
101	Spring Field	1001	North Field Zone	4

Update data

```sql
-- Update customer_email
UPDATE customers
SET customer_email= 'info_company@greenfarm.'
WHERE customer_id = 2;
```





Sample Queries for answer business questions

1. Farm data sensor information full scope

```sql
SELECT
    c.customer_id AS customer_id,
    c.customer_name AS customer_name,
    f.farm_id AS farm_id,
    f.farm_name AS farm_name,
    z.zone_id AS zone_id,
    z.zone_name AS zone_name,
    s.sensor_id AS sensor_id,
    s.sensor_name AS sensor_name,
    sd.data_id AS data_id,
    sd.timestamp AS sensor_data_timestamp,
    sd.value AS sensor_data_value,
    asr.sensor_rule_id AS sensor_rule_id,
    asr.rule_name AS rule_name,
    sr.sensors_rule_id AS sensors_rule_id
FROM customers c
LEFT JOIN farms f ON c.customer_id = f.customer_id
LEFT JOIN zones z ON f.farm_id = z.farm_id
LEFT JOIN sensors s ON z.zone_id = s.zone_id
LEFT JOIN sensor_data sd ON s.sensor_id = sd.sensor_id
LEFT JOIN sensors_rule sr ON s.sensor_id = sr.sensor_id
LEFT JOIN alert_sensor_rule asr ON sr.sensor_rule_id = asr.sensor_rule_id
LEFT JOIN alert_coming ac ON sr.sensors_rule_id = ac.sensor_rule_id;
```

stomers(+) 1

ELECT c.customer_id AS customer_id, c.customer_name AS customer_n | Enter a SQL expression to filter results (use Ctrl+Space)

customer_id	customer_name	farm_id	farm_name	zone_id	zone_name	sensor_id	sensor_name	data_id	sensor_data_timestamp	sensor_data_value	sensor_rule_id	rule
2	GreenFarm Inc	101	Spring Field	1,001	North Field Zone	2,001	Soil Moisture Sensor A	2	2025-01-16 15:49:46	40.2	3,001	High M
2	GreenFarm Inc	101	Spring Field	1,001	North Field Zone	2,001	Soil Moisture Sensor A	1	2025-01-17 15:49:46	35	3,001	High M

Sample Queries for answer business questions

2. Which farms are currently registered in the system ?

```sql
-- Which farms are currently registered in the system?
SELECT farm_id, farm_name, owner_name, supervisor, location, size
FROM farms;
```

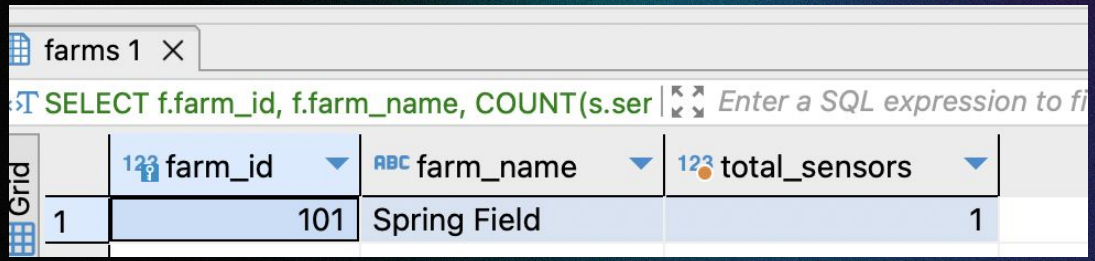
T farm_id, farm_name, owner_name, supervisor, location, size FROM | Enter a SQL expression to filter results (use Ctrl+Space)

farm_id	farm_name	owner_name	supervisor	location	size
101	Spring Field	Khoa	ABC	Riverside Area	120

Sample Queries for answer business questions

3. Which farm and zone that has the most sensors installed?

```sql
-- which farm and zone in that farm has the most sensors installed?
SELECT f.farm_id,
       f.farm_name,
       z.zone_id,
       z.zone_name,
       COUNT(s.sensor_id) AS total_sensors
FROM farms f
LEFT JOIN sensors s ON f.farm_id = s.farm_id
LEFT JOIN zones z ON f.farm_id = z.farm_id
GROUP BY f.farm_id, f.farm_name, z.zone_id, z.zone_name
ORDER BY total_sensors DESC
LIMIT 1;
```



	farm_id	farm_name	total_sensors
1	101	Spring Field	1

Sample Queries for answer business questions

4. Which farm and zone that has the highest Moisture value from the last 48 hours?

```sql
-- Farm and zone has highest Moisture from the last 48 hours
SELECT f.farm_id,
       f.farm_name,
       z.zone_id,
       z.zone_name,
       MAX(sd.value) AS max_moisture
FROM farms f
JOIN sensors s
    ON f.farm_id = s.farm_id
JOIN sensor_data sd
    ON s.sensor_id = sd.sensor_id
JOIN zones z
    ON z.farm_id = f.farm_id
WHERE s.sensor_type = 'Moisture'
  AND sd.timestamp >= DATE_SUB(NOW(), INTERVAL 48 HOUR)
GROUP BY f.farm_id, f.farm_name, z.zone_id, z.zone_name
ORDER BY max_moisture DESC
LIMIT 1;
```

(+) 1

CT f.farm_id, f.farm_name, z.zone_id, z.z | Enter a SQL expression to filter results (use Ctrl+Space)

farm_id	farm_name	zone_id	zone_name	max_moisture
101	Spring Field	1,001	North Field Zone	40.2

Farm and zone has highest Moisture from the last 48 hours

North Field Zone farm_name	Spring Field zone_name	Spring Field North Field Zone
Spring Field	North Field Zone	40.20%

Sample Queries for answer business questions

5. Rolling average 1h sensor data to track the trend and smoothen the curve

```sql
-- rolling avg 1h of a sensor
SELECT
    sensor_id,
    timestamp,
    value,
    ROUND(AVG(value) OVER (
      PARTITION BY sensor_id
      ORDER BY timestamp
      RANGE BETWEEN INTERVAL 1 HOUR PRECEDING AND CURRENT ROW
    ),2) AS rolling_1h_avg
FROM sensor_data
WHERE sensor_id = 2001
ORDER BY timestamp;
```

```sql
-- add some more data for calculating rolling average 1 hour
INSERT INTO sensor_data (sensor_id, timestamp, value)
VALUES
    (2001, NOW() - INTERVAL 2 HOUR, 33.0),
    (2001, NOW() - INTERVAL 105 MINUTE, 34.5),
    (2001, NOW() - INTERVAL 90 MINUTE, 36.2),
    (2001, NOW() - INTERVAL 75 MINUTE, 37.0),
    (2001, NOW() - INTERVAL 1 HOUR, 38.5),
    (2001, NOW() - INTERVAL 45 MINUTE, 37.9),
    (2001, NOW() - INTERVAL 30 MINUTE, 39.1),
    (2001, NOW() - INTERVAL 15 MINUTE, 40.2),
    (2001, NOW(), 39.0);
```

_data 1

T sensor_id, timestamp, value, ROUND(AVG(value) (Enter a SQL expression to filter results (use Ctrl+Space)

sensor_id	timestamp	value	rolling_1h_avg
2,001	2025-01-16 15:49:46	40.2	40.2
2,001	2025-01-17 00:49:51	32.1	32.1
2,001	2025-01-17 01:49:51	31.5	31.8
2,001	2025-01-17 02:49:51	29.8	30.65
2,001	2025-01-17 03:49:51	28.9	29.35
2,001	2025-01-17 04:49:51	27.3	28.1
2,001	2025-01-17 05:49:51	26.7	27
2,001	2025-01-17 06:49:51	27.1	26.9
2,001	2025-01-17 07:49:51	29.4	28.25
2,001	2025-01-17 08:49:51	30.2	29.8
2,001	2025-01-17 09:49:51	31.6	30.9
2,001	2025-01-17 10:49:51	33.1	32.35
2,001	2025-01-17 11:49:51	35	34.05
2,001	2025-01-17 12:49:51	36.4	35.7
2,001	2025-01-17 13:49:51	37.2	36.8
2,001	2025-01-17 14:49:51	33	35.1
2,001	2025-01-17 15:04:51	34.5	33.75
2,001	2025-01-17 15:19:51	36.2	34.57



Sample Queries for answer business questions

6. Count number of alert per farm

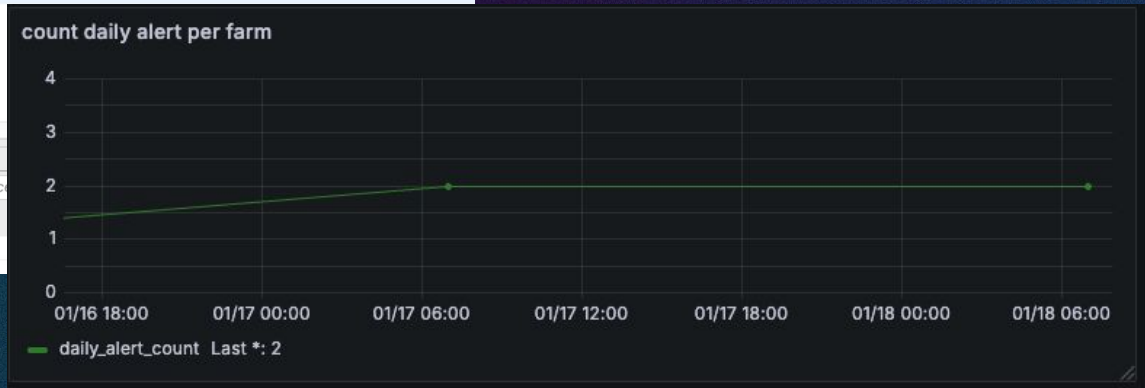
```sql
-- count daily alert per farm
SELECT DATE(ac.alert_timestamp) AS alert_date,
       f.farm_id,
       f.farm_name,
       COUNT(ac.alert_id) AS daily_alert_count
FROM alert_coming ac
left JOIN sensors_rule sr ON ac.sensor_rule_id = sr.sensor_rule_id
left JOIN sensors s ON sr.sensor_id = s.sensor_id
left JOIN alert_sensor_rule asr ON sr.sensor_rule_id = asr.sensor_rule_id
left JOIN farms f
    ON s.farm_id = f.farm_id
GROUP BY DATE(ac.alert_timestamp),
         f.farm_id,
         f.farm_name
ORDER BY alert_date ASC, f.farm_id;


-- Farm has highest Moisture
```

ns 1 ×

ECT DATE(ac.alert_timestamp) AS alert_date, f.farm_i | Enter a SQL expression to filter results (use Ctrl+Spac

alert_date	farm_id	farm_name	daily_alert_count
2025-01-17	101	Spring Field	1



Sample Queries for answer business questions

7. CO2 data percentile

```sql
-- percentile CO2 data
WITH cte AS (
    SELECT
        value,
        ROW_NUMBER() OVER (ORDER BY value) AS rn,
        COUNT(*) OVER () AS total_rows
    FROM sensor_data
    Where sensor_id = 5001
)
SELECT
    MAX(CASE WHEN rn = FLOOR(0.25 * (total_rows + 1)) THEN value END)
    MAX(CASE WHEN rn = FLOOR(0.50 * (total_rows + 1)) THEN value END)
    MAX(CASE WHEN rn = FLOOR(0.75 * (total_rows + 1)) THEN value END)
FROM cte;
```

1 ×

cte AS (SELECT value, ROW_NUMBER() OVER (ORD | Enter a SQL expression to filter results (use Ctrl+Spac

123 percentile_25	123 percentile_50	123 percentile_75
410.2	445.3	460.2



Sample Queries for answer business questions

8. Day over Day alert count difference trend of a farm

```sql
With daily_alerts AS (
SELECT
      DATE(ac.alert_timestamp) AS alert_date,
      f.farm_id,
      f.farm_name,
      COUNT(ac.alert_id) AS daily_alert_count
  FROM alert_coming ac
left JOIN sensors_rule sr ON ac.sensor_rule_id = sr.sensor_rule_id
left JOIN sensors s ON sr.sensor_id = s.sensor_id
left JOIN alert_sensor_rule asr ON sr.sensor_rule_id = asr.sensor_rule_id
 left JOIN farms f          ON s.farm_id = f.farm_id
  GROUP BY DATE(ac.alert_timestamp), f.farm_id, f.farm_name)

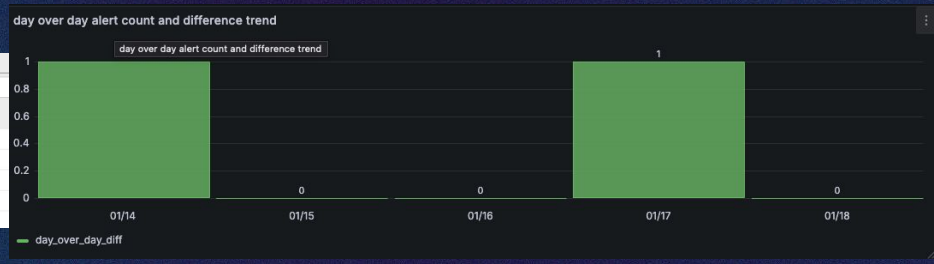
 SELECT
  alert_date,
  farm_id,
  farm_name,
  daily_alert_count,
  LAG(daily_alert_count, 1, 0) OVER (PARTITION BY farm_id ORDER BY alert_date) AS previous_day_cou
  (daily_alert_count
     - LAG(daily_alert_count, 1, 0) OVER (PARTITION BY farm_id ORDER BY alert_date))
       AS day_over_day_diff
FROM daily_alerts
ORDER BY farm_id, alert_date;
```

```sql
-- add more dummy alert to this farm for calculating day over day alert difference
INSERT INTO alert_coming (alert_id, sensor_rule_id, alert_timestamp)
VALUES
  -- A few days old
  (5002, 3001, NOW() - INTERVAL 3 DAY),
  (5003, 3001, NOW() - INTERVAL 2 DAY),
  (5004, 3001, NOW() - INTERVAL 1 DAY),
  (5005, 3001, NOW() - INTERVAL 2 HOUR);
```

farms 1

With daily_alerts AS (SELECT DATE(ac.alert_timestamp) ... Enter a SQL expression to filter results (use Ctrl+Space)

	alert_date	farm_id	farm_name	daily_alert_count	previous_day_count	day_over_day_diff
1	2025-01-14	101	Spring Field	1	0	1
2	2025-01-15	101	Spring Field	1	1	0
3	2025-01-16	101	Spring Field	1	1	0
4	2025-01-17	101	Spring Field	2	1	1



Sample Queries for answer business questions

9. Energy consumption per shift

```sql
-- energy consumption of a sensor
WITH cte_labeled AS (
  SELECT
    sd.sensor_id,
    sd.timestamp,
    sd.value,
    CASE
      WHEN (HOUR(sd.timestamp) >= 6 AND HOUR(sd.timestamp) < 14)
           THEN 'Day'
      WHEN (HOUR(sd.timestamp) >= 14 AND HOUR(sd.timestamp) < 22)
           THEN 'Afternoon'
      ELSE 'Night'
    END AS shift_label
  FROM sensor_data sd
  WHERE sd.sensor_id = 3001  -- ID of your energy sensor
)
SELECT
  DATE(timestamp) AS usage_date,
  shift_label,
  MIN(value) AS shift_start_value,
  MAX(value) AS shift_end_value,
  (MAX(value) - MIN(value)) AS usage_kwh
FROM cte_labeled
GROUP BY DATE(timestamp), shift_label
ORDER BY
  usage_date,
  FIELD(shift_label, 'Day','Afternoon','Night');
```



s 1 ×

cte_labeled AS (SELECT sd.sensor_id, sd.timestamp | Enter a SQL expression to filter results (use Ctrl+Space)

usage_date	shift_label	shift_start_value	shift_end_value	usage_kwh
2025-01-18	Day	1,000	1,110	110
2025-01-18	Afternoon	1,112	1,200	88
2025-01-18	Night	1,205	1,205	0
2025-01-19	Night	1,250	1,290	40

Sample Queries for answer business questions

10. Pareto Alert Analysis

```sql
-- 80/20 Pareto alert investigation
WITH sensor_alerts AS (
    SELECT
        s.sensor_id,
        s.sensor_name,
        z.zone_id,
        z.zone_name,
        COUNT(*) AS alert_count
    FROM alert_coming ac
left JOIN sensors_rule sr ON ac.sensor_rule_id = sr.sensor_rule_id
left JOIN sensors s ON sr.sensor_id = s.sensor_id
left JOIN alert_sensor_rule asr ON sr.sensor_rule_id = asr.sensor_rule_id
left JOIN farms f          ON s.farm_id = f.farm_id
LEFT JOIN zones z ON f.farm_id = z.farm_id
   GROUP BY
        s.sensor_id,
        s.sensor_name,
        z.zone_id,
        z.zone_name
),
total_alerts AS (
    SELECT SUM(alert_count) AS grand_total
    FROM sensor_alerts
)
SELECT
    sa.sensor_id,
    sa.sensor_name,
    sa.zone_id,
    sa.zone_name,
    sa.alert_count,
    ROUND(sa.alert_count / ta.grand_total * 100, 2) AS perc_of_total,
    ROUND(
      SUM(sa.alert_count) OVER (
        ORDER BY sa.alert_count DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
      )
      / ta.grand_total * 100,
      2
    ) AS cume_perc_of_total
FROM sensor_alerts sa
CROSS JOIN total_alerts ta
ORDER BY
    sa.alert_count DESC;
```

	sensor_id	sensor_name	zone_id	zone_name	alert_count	perc_of_total	cume_perc_of_total
1	2,001	Soil Moisture Sensor A	1,001	North Field Zone	5	71.43	71.43
2	4,001	Wind Speed Sensor	1,001	North Field Zone	2	28.57	100

